

Image Classification using Convolutional Neural Networks (CNNs) and MLP on Real-World Scene Images

Baitursyn Arna

NARXOZ
UNIVERSITY

Problem and Dataset

buildings



forest



glacier



mountain



sea



street



I used the Intel Image Classification dataset from Kaggle.

It contains about 25,000 real-world images.

All images are resized to 150 by 150 pixels.

The goal of this project is to classify images into 6 categories: buildings, forest, mountains, sea, glacier, and street.

Model Approach

In this project, I used two different models:

1. Convolutional Neural Network (CNN)

The first model is a CNN.

CNN is very powerful for image classification because it can automatically learn features from images.

It learns step by step:

- first edges
- then shapes
- then more complex patterns

My CNN model includes:

- convolutional layers for feature extraction
- max pooling layers for reducing image size
- batch normalization for stable training
- dropout to reduce overfitting
- dense layers for final classification

2. Multilayer Perceptron (MLP)

The second model is MLP.

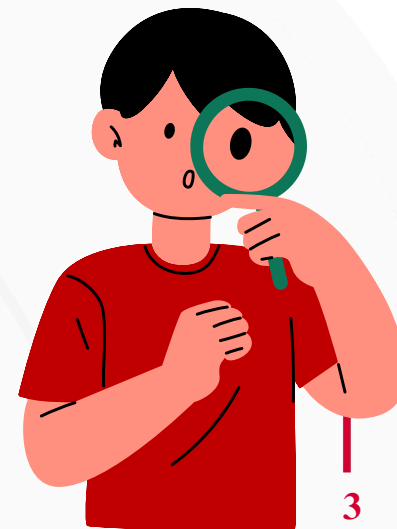
MLP is a simple neural network that uses fully connected layers.

Before entering the model, images are flattened into a one-dimensional vector.

The problem is that MLP does not understand spatial structure of images.

So it loses important visual information.

That is why MLP is used only as a baseline model.

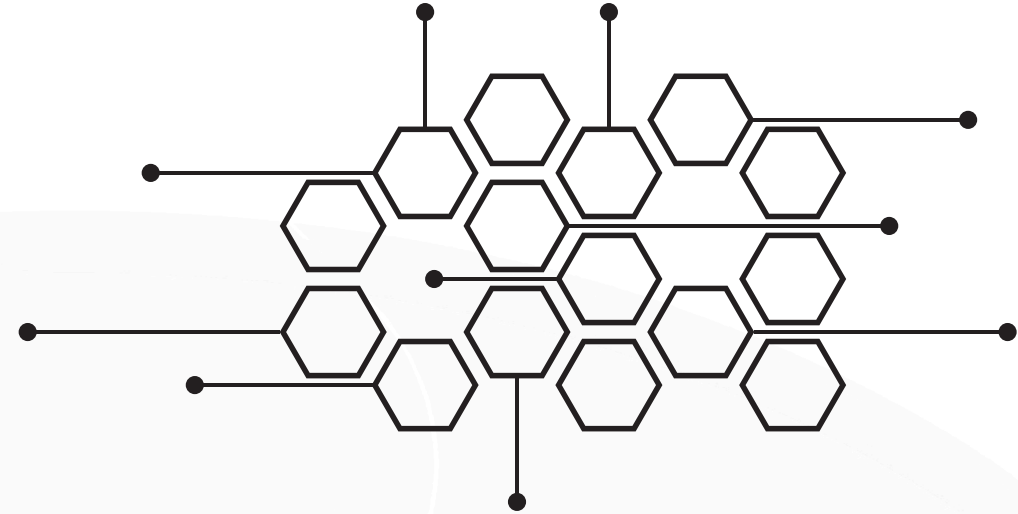
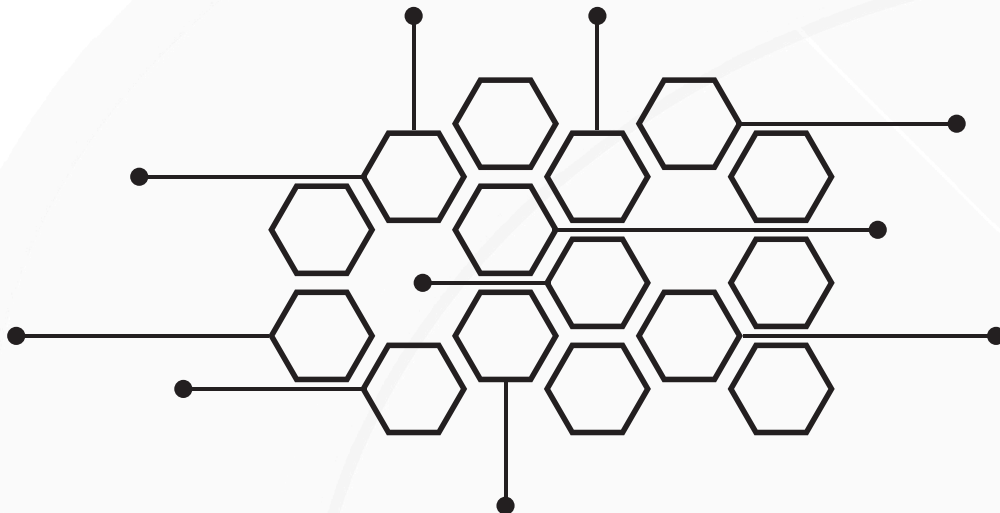


Batch Normalization and Training Setup

I also used Batch Normalization in the CNN model.

It helps to:

- make training faster
- stabilize learning
- reduce internal changes in data distribution
- improve final performance



For training both models, I used:

- loss function: Cross-Entropy Loss
- optimizer: Adam
- batch size: 32
- image size: 150×150
- train/validation/test split: 80% / 10% / 10%
- data augmentation: flipping, rotation, zooming

Data augmentation helps the model learn better and reduces overfitting.



Main Results

After training both models, I compared their performance.

Results:

- CNN accuracy: about 78% to 85%
- MLP accuracy: about 32% to 45%
-

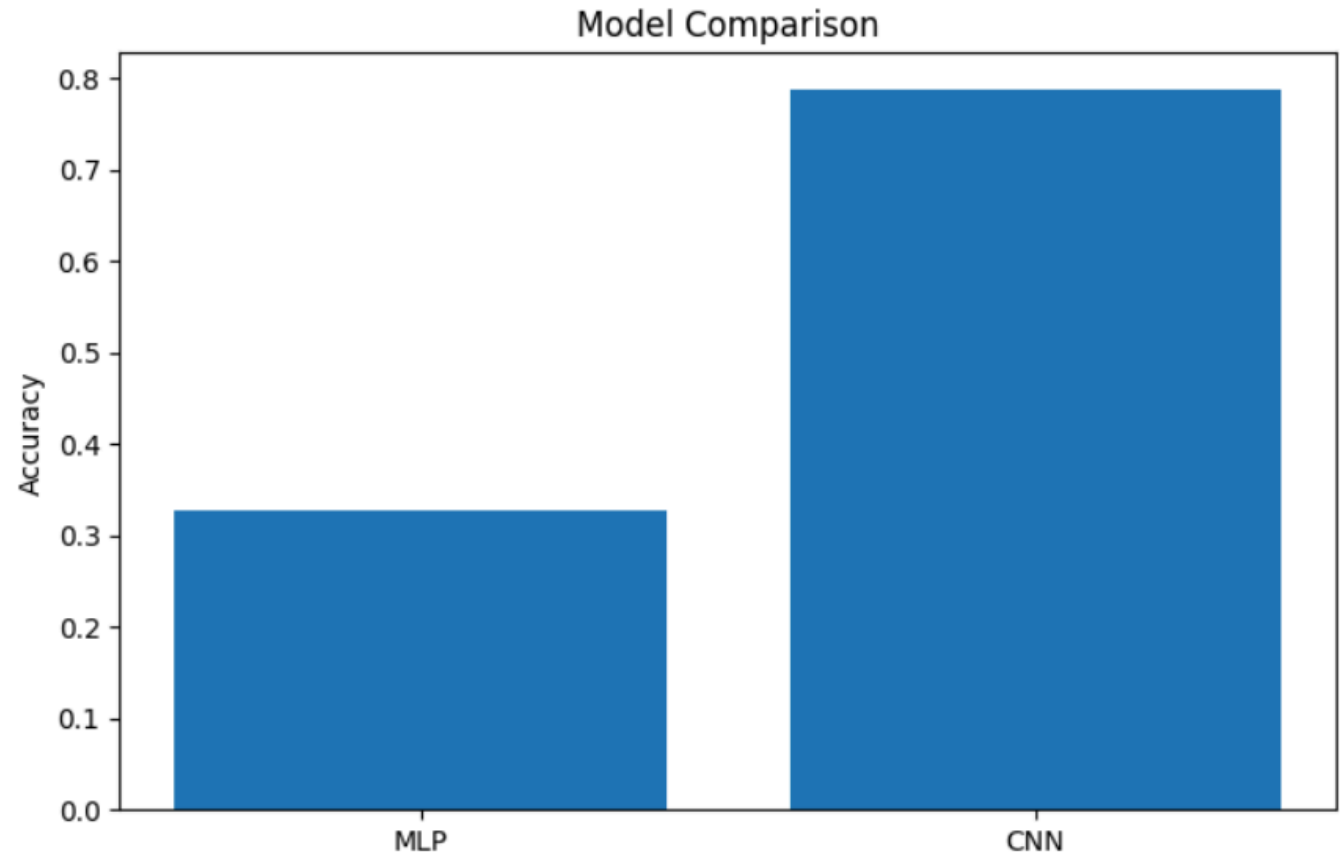
Comparison:

CNN performed much better than MLP.

The main reason is that CNN understands spatial features in images, while MLP does not. CNN also generalizes better on unseen data.

MLP is faster, but not suitable for image classification tasks.

CNN Accuracy: 0.7883333563804626
MLP Accuracy: 0.32733333110809326



Example of Success

One successful example was when the model correctly predicted a “buildings” image.

The image contained houses and man-made structures.

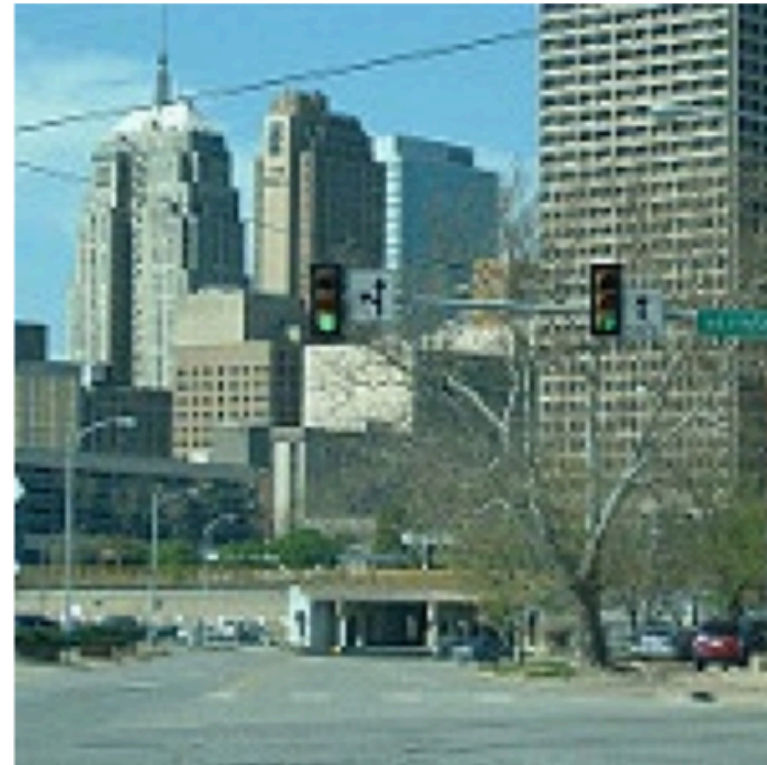
The CNN model recognized it correctly with high confidence.

This shows that CNN is good at detecting shapes and structural patterns in urban scenes.

True: buildings
Pred: buildings



True: buildings
Pred: buildings



Example of Failure

One wrong prediction happened between “buildings” and “street”. These two classes are very similar because both are part of urban scenes.

In some images, buildings and streets appear together, which makes it difficult to distinguish them.

That is why the model sometimes makes mistakes in these cases.

True: buildings
Pred: street



True: buildings
Pred: mountain



These two classes can sometimes look similar because buildings often appear near mountains or natural landscapes.

In some images, the background and shapes can confuse the model.

That is why the model sometimes makes mistakes in these cases.

What I would improve

If I continue this project, I would improve it in several ways:

- use a more advanced model like ResNet or VGG
- apply transfer learning from pre-trained models
- train for more epochs to improve accuracy
- improve data augmentation techniques
- try hyperparameter tuning

These improvements can increase accuracy and reduce errors.



Conclusion

In conclusion, CNN is much better than MLP for image classification tasks.

CNN can learn important visual features from images, while MLP cannot.

Batch Normalization and data augmentation also improved model performance.

This project helped me understand how deep learning works in real computer vision problems.

Thank you for listening.



**THANK YOU
FOR YOUR ATTENTION.**